# Sécuriser un dépôt Python avec les hooks `pre-commit`

**Module :** Pilotage & amélioration continue de la sécurité
**Durée indicative :** 2h
**Niveau :** intermédiaire
**Environnement :** Python, Git, `pre-commit`, Bandit, `detect-secrets`, `pytest`, `pip-audit`
**Projet fil rouge :** NOVA SERVICES
**Thème :** intégrer des contrôles de qualité et de sécurité dès la phase de développement.

## Introduction — Pourquoi sécuriser le code avant le commit ?

Dans une équipe de développement, certaines erreurs peuvent être évitées avant même que le code ne soit envoyé vers le dépôt distant :

- Un développeur ajoute accidentellement un mot de passe dans un fichier Python.
- Une fonction utilise `eval()` sur une donnée contrôlée par un utilisateur.
- Un fichier contient des erreurs de formatage qui compliquent la revue de code.
- Une dépendance Python utilisée par l'application présente une vulnérabilité connue.
- Une modification casse des tests unitaires.

Ces problèmes peuvent être détectés par différents contrôles automatisés.

L'objectif de ce TP est de mettre en place des **vérifications automatiques avant chaque commit**, puis de reprendre ces vérifications dans la chaîne d'intégration continue (CI).

Le framework `pre-commit` permet de configurer et d'exécuter plusieurs outils de contrôle dans un même dépôt Git. https://pre-commit.com/

### Objectifs pédagogiques

À la fin du TP, vous devez savoir :

- Installer et configurer `pre-commit`.
- Détecter les secrets présents dans le code.
- Analyser du code Python avec Bandit.
- Corriger des problèmes de qualité et de sécurité.
- Exécuter des tests unitaires avant le commit.
- Ajouter une vérification des dépendances vulnérables.

- Reproduire les contrôles en CI.
- Distinguer les contrôles locaux des règles imposées par le serveur Git.
- Définir des indicateurs permettant de suivre l'efficacité des contrôles.

# 1. Contexte professionnel

Vous travaillez au sein de l'équipe de développement de NOVA SERVICES. L'entreprise développe une API Python utilisée par ses applications internes et ses clients.

Le responsable sécurité constate plusieurs problèmes dans les dépôts :

- Des secrets sont parfois ajoutés dans les fichiers de configuration.
- Les revues de code détectent tardivement des pratiques dangereuses.
- Les tests ne sont pas systématiquement exécutés avant les commits.
- Les dépendances ne sont pas toujours vérifiées.
- Les règles de sécurité varient selon les développeurs.

Votre mission consiste à proposer une première chaîne de sécurisation du code.

### User Story — US-DEVSEC-001

**En tant que** responsable sécurité,
**je veux** que les contrôles élémentaires de qualité et de sécurité soient exécutés automatiquement avant les commits,
**afin de** réduire les erreurs introduites dans le dépôt et d'améliorer la qualité du code livré.

### Critères d'acceptation

- Les hooks sont versionnés dans le dépôt.
- Un commit contenant un problème détecté par un contrôle bloquant échoue.
- Les développeurs disposent d'instructions reproductibles.
- Les contrôles peuvent être exécutés sur l'ensemble du dépôt.
- La CI reprend les contrôles essentiels.
- Les exceptions sont justifiées et traçables.

# 2. Prérequis techniques

Vérifiez que Python, `pip` et Git sont disponibles :

```
python --version
python -m pip --version
git --version
```

Les commandes peuvent nécessiter `python3` à la place de `python`, selon votre environnement.

Créez un répertoire de travail :

mkdir pre-commit
cd pre-commit

git init

Créez un environnement virtuel Python :

python -m venv .venv

Activation sous Linux ou macOS :

source .venv/bin/activate

Activation sous Windows PowerShell :

.\.venv\Scripts\Activate.ps1

Installez les outils de travail :

python -m pip install --upgrade pip
python -m pip install pre-commit bandit detect-secrets pytest pip-audit

Vérifiez l'installation :

pre-commit --version
bandit --version
detect-secrets --version
pytest --version
pip-audit --version

**Bonne pratique :** pour un projet d'équipe, documentez les versions de Python et des outils utilisés, et reproduisez l'environnement dans la CI.

# 3. Créer un code volontairement imparfait

Créez le fichier `app.py` suivant :

```
import os

def get_message():
```

```
    password = "demo-secret-123"
    user_input = input("Entrez une expression : ")
    result = eval(user_input)
    return f"Résultat : {result}"
```

Ce code est volontairement imparfait. La valeur de démonstration est fictive : ne remplacez jamais cet exemple par un véritable secret.

### Travail demandé

1. Identifiez les problèmes de qualité du code.
2. Identifiez les risques de sécurité.
3. Expliquez pourquoi `eval()` est dangereux lorsqu'il est utilisé sur une entrée non fiable.
4. Expliquez pourquoi un mot de passe ne doit pas être stocké dans le code source.
5. Proposez une première correction, sans installer encore de hook.

**Point important :** un scanner de secrets cherche des motifs suspects et peut produire des faux positifs. Un outil d'analyse statique ne détecte pas non plus toutes les vulnérabilités. Les contrôles doivent être combinés à la revue de code et aux tests.

## 4. Configurer les contrôles de base avec `pre-commit`

À la racine du dépôt, créez le fichier `.pre-commit-config.yaml` :

```
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-added-large-files
      - id: check-yaml

  - repo: https://github.com/PyCQA/bandit
    rev: 1.8.6
    hooks:
      - id: bandit
        args: ["-ll"]

  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks:
      - id: detect-secrets
        args: ["--baseline", ".secrets.baseline"]
```

```
default_language_version:
  python: python3
```

Les versions ci-dessus sont des exemples épinglés pour rendre la configuration reproductible. Vérifiez leur disponibilité et leur compatibilité avec votre environnement, puis faites évoluer les versions de manière contrôlée.

Les contrôles configurés servent à :

| Contrôle | Utilité |
|---|---|
| `trailing-whitespace` | Supprimer les espaces inutiles en fin de ligne. |
| `end-of-file-fixer` | Garantir une fin de fichier correcte. |
| `check-added-large-files` | Éviter l'ajout accidentel de fichiers volumineux. |
| `check-yaml` | Vérifier la syntaxe des fichiers YAML. |
| `bandit` | Repérer certaines pratiques Python potentiellement dangereuses. |
| `detect-secrets` | Repérer des secrets potentiels dans les fichiers. |

La configuration est partagée dans le dépôt : chaque membre de l'équipe peut ainsi utiliser les mêmes règles.

## 4.1. Initialiser le baseline de détection des secrets

`detect-secrets` peut utiliser un fichier de référence, appelé *baseline*, pour répertorier les éléments suspects déjà connus et permettre leur audit.

Pour créer un premier baseline :

detect-secrets scan > .secrets.baseline

Le code volontairement imparfait peut faire apparaître des résultats dans ce fichier. Examinez-les et identifiez les éventuels faux positifs.

Le baseline ne doit pas servir à masquer un vrai secret. Un secret réel découvert dans le dépôt doit être retiré du code et, s'il a été exposé, révoqué ou renouvelé. La création d'un baseline ne remplace pas ce traitement.

Documentation : https://github.com/Yelp/detect-secrets

## 4.2. Installer les hooks dans Git

Exécutez :

pre-commit install

L'installation ajoute le mécanisme qui lance les hooks lors des commits effectués depuis ce dépôt.

Vérifiez ensuite la configuration sur tous les fichiers :

pre-commit run --all-files

Certains hooks corrigent automatiquement les fichiers. Si cela se produit, examinez les modifications :

git diff

Puis relancez les contrôles après correction.

## 4.3. Tester le premier commit

git add .
git commit -m "test: ajout des contrôles pre-commit"

Observez les résultats.

Un commit peut être refusé si un hook échoue. Cela ne signifie pas que Git est cassé : c'est le mécanisme de prévention recherché.

Si un hook modifie automatiquement un fichier, ajoutez de nouveau la version corrigée :

git add .
git commit -m "test: correction des contrôles"

**À retenir :** un hook local est utile, mais il peut être contourné, par exemple avec `git -c core.hooksPath=/dev/null commit ...`. La CI et les règles de protection du dépôt distant sont donc nécessaires pour compléter le dispositif.

# 5. Corriger les problèmes détectés

## 5.1. Retirer le secret du code

Ne stockez pas de mot de passe directement dans `app.py`.

Créez un fichier `.env` pour votre environnement local :

APP_PASSWORD=valeur-fictive-pour-le-tp

Ajoutez à `.gitignore` :

.venv/
__pycache__/
.pytest_cache/
.env

Créez également un fichier `.env.example` qui ne contient aucune valeur secrète :

APP_PASSWORD=

Le fichier `.env` reste local et ne doit pas être ajouté au dépôt.

Si un fichier `.env` a déjà été indexé par Git, retirez-le de l'index :

git rm --cached .env

Cette commande ne révoque pas un secret déjà publié et ne supprime pas son éventuelle présence dans l'historique. Si un vrai secret a été exposé, il faut le révoquer ou le renouveler, puis traiter l'historique si nécessaire.

## 5.2. Charger la configuration sans afficher le secret

Installez `python-dotenv` :

python -m pip install python-dotenv

Dans `app.py` :

```
import os

from dotenv import load_dotenv

load_dotenv()


def get_password():
    password = os.getenv("APP_PASSWORD")
    if not password:
        raise RuntimeError("APP_PASSWORD n'est pas configuré")
    return password
```

Ne faites pas de `print(password)` et ne l'incluez pas dans les logs.

**Attention :** une variable d'environnement réduit le risque d'exposer un secret dans le code source, mais ne constitue pas à elle seule un coffre-fort. En production, privilégiez un gestionnaire de secrets adapté à l'environnement de déploiement.

## 5.3. Remplacer l'utilisation dangereuse de `eval()`

Pour une application qui doit calculer une opération simple, il est préférable de définir explicitement les opérations autorisées plutôt que d'évaluer une expression arbitraire.

Exemple :

```
def calculate(a: float, b: float, operation: str) -> float:
    if operation == "add":
        return a + b
    if operation == "subtract":
        return a - b
    if operation == "multiply":
        return a * b
    if operation == "divide":
        if b == 0:
            raise ValueError("Division par zéro")
        return a / b

    raise ValueError("Opération non autorisée")
```

Cette approche limite les opérations possibles et rend le comportement plus facile à tester.

Relancez les contrôles :

```
pre-commit run --all-files
```

Corrigez les erreurs jusqu'à obtenir un résultat satisfaisant. Si `detect-secrets` signale une valeur fictive, examinez le résultat et configurez le baseline ou les exceptions uniquement après avoir vérifié qu'il ne s'agit pas d'un secret réel.

# 6. Ajouter des tests unitaires aux hooks

Les tests permettent de vérifier que les corrections ne dégradent pas le fonctionnement de l'application.

Créez `test_app.py` :

```
import pytest

from app import calculate


def test_addition():
    assert calculate(2, 3, "add") == 5


def test_subtraction():
    assert calculate(5, 2, "subtract") == 3


def test_division_by_zero():
    with pytest.raises(ValueError):
        calculate(10, 0, "divide")


def test_unknown_operation():
    with pytest.raises(ValueError):
        calculate(1, 2, "power")
```

Vérifiez les tests :

pytest -q

Ajoutez ensuite un hook local dans `.pre-commit-config.yaml` :

```
 - repo: local
    hooks:
      - id: pytest
        name: Tests unitaires
        entry: pytest -q
        language: system
        pass_filenames: false
        always_run: true
```

Placez ce bloc au même niveau que les autres entrées de **`repos`**, en respectant l'indentation YAML.

Ce hook suppose que `pytest` est installé dans l'environnement actif. Dans une équipe, utilisez un environnement de projet reproductible et installez les dépendances de test avant d'exécuter les hooks.

Testez :

pre-commit run pytest --all-files

Puis essayez de modifier temporairement un test pour le faire échouer. Le commit doit être bloqué tant que les tests échouent.

**Discussion :** exécuter toute la suite de tests avant chaque commit peut ralentir les développeurs. Pour un grand projet, on peut réserver les tests rapides aux hooks locaux et exécuter les suites plus longues en CI.

# 7. Ajouter un contrôle des dépendances vulnérables

Un code propre peut tout de même dépendre d'une bibliothèque vulnérable.

L'outil `pip-audit` vérifie les dépendances Python déclarées et recherche les vulnérabilités connues associées aux paquets et versions identifiés.

## 7.1. Déclarer les dépendances

Créez un fichier `requirements.txt`, par exemple :

python-dotenv

Créez aussi `requirements-dev.txt` pour les outils de développement :

-r requirements.txt
pre-commit
bandit
detect-secrets
pytest
pip-audit

Dans un vrai projet, il est préférable de verrouiller les versions et de reproduire l'environnement utilisé par les développeurs et la CI.

### 7.2. Exécuter l'audit

pip-audit -r requirements.txt

Examinez les résultats et notez :

- Le paquet concerné.
- La version installée ou résolue.
- L'identifiant de vulnérabilité, lorsqu'il est fourni.
- La version corrigée disponible, si elle existe.
- L'impact potentiel sur l'application.
- L'action envisagée.

### 7.3. Ajouter le contrôle à `pre-commit`

Ajoutez un deuxième hook local :

```
  - id: pip-audit
    name: Audit des dépendances Python
    entry: pip-audit -r requirements.txt
    language: system
    pass_filenames: false
    always_run: true
```

Ce hook s'exécute à chaque commit. Il suppose que `pip-audit` est installé et que le fichier `requirements.txt` existe.

Dans un projet réel, décidez si cet audit doit être bloquant localement ou seulement en CI. La disponibilité du service de vulnérabilités, la durée d'exécution et le risque associé aux dépendances doivent être pris en compte.

**Limite importante :** `pip-audit` ne prouve pas que l'application est sûre. Il identifie des vulnérabilités connues dans les dépendances analysées ; il ne remplace ni les tests, ni l'analyse du code, ni l'évaluation du risque métier.

# 8. Vérifier le fonctionnement complet

Une fois la configuration terminée, exécutez :

```
pre-commit run --all-files
pytest -q
pip-audit -r requirements.txt
```

Puis préparez un commit :

```
git status
```

git add .
git commit -m "security: ajout des contrôles pre-commit"

### Scénarios de validation

| Scénario | Résultat attendu |
|---|---|
| Un fichier comporte des espaces en fin de ligne | Le hook les corrige ou signale une modification à ajouter de nouveau. |
| Un fichier contient un secret suspect | **`detect-secrets`** signale le résultat à examiner. |
| Du code utilise une fonction jugée risquée | Bandit peut signaler le problème selon ses règles. |
| Un test unitaire échoue | Le hook de test échoue et bloque le commit. |
| Une dépendance présente une vulnérabilité connue | `pip-audit` signale la vulnérabilité ; le hook échoue selon le résultat de l'audit. |
| Tous les contrôles réussissent | Le commit peut être créé. |

Les résultats exacts dépendent des versions des outils, des fichiers analysés et de leur configuration. Ne considérez pas un contrôle réussi comme une preuve absolue d'absence de vulnérabilité.

# 9. Reprendre les contrôles dans la CI (optionnel)

Un hook local ne suffit pas : un développeur peut ne pas l'avoir installé, ou contourner son exécution. La CI doit donc refaire les contrôles. Vous pouvez également utiliser Github Action.

Exemple de workflow GitLab:

Les actions GitLab référencées ici sont des exemples de versions majeures. Dans un environnement de production, épinglez les actions à des versions ou des SHA validés par l'organisation et entretenez-les régulièrement.

## 9.1. Bloquer les commits directs sur `main`

Un hook local ne peut pas garantir qu'aucun développeur ne pousse directement sur `main`.

Pour imposer une règle d'équipe, configurez dans l'hébergement Git :

- Une protection de branche ou une règle de dépôt.
- L'interdiction des pushes directs si le processus l'exige.
- La création de pull requests.
- L'obligation de réussir les vérifications CI.
- Une approbation de revue de code, selon la politique de l'entreprise.

La règle doit être appliquée côté serveur, et non seulement documentée dans le dépôt local.

### 9.2. Centraliser la configuration avec `pre-commit.ci`

`pre-commit.ci` peut exécuter les hooks sur les pull requests, appliquer certains correctifs et proposer des mises à jour de configuration. Le service est notamment documenté pour les dépôts hébergés sur GitHub : https://pre-commit.ci/.

Il complète le dispositif, mais ne remplace pas les règles de protection de branche, les tests applicatifs ou une politique de gestion des vulnérabilités.

## 10. Exercice complémentaire — Adapter les contrôles à NOVA SERVICES

L'équipe souhaite appliquer ces pratiques à plusieurs dépôts : une API Python, un outil d'administration et des scripts d'automatisation.

Proposez une configuration adaptée en répondant aux questions suivantes :

1. Quels contrôles doivent être communs à tous les dépôts ?
2. Quels contrôles doivent être spécifiques à Python ?
3. Quels contrôles doivent être bloquants avant le commit ?
4. Quels contrôles peuvent être exécutés uniquement en CI ?
5. Comment éviter qu'une exception à `detect-secrets` masque un véritable secret ?
6. Qui est responsable de la mise à jour des versions des hooks ?
7. Comment prouver qu'un développeur a corrigé une vulnérabilité ?
8. Comment traiter une dépendance vulnérable pour laquelle aucun correctif n'est disponible ?

### Livrable complémentaire

Rédigez une courte politique de sécurisation du code qui précise :

- Les contrôles obligatoires.
- Les contrôles exécutés localement et en CI.
- Les conditions de blocage.
- Le traitement des faux positifs.
- Le responsable de la validation des exceptions.
- La procédure de mise à jour des outils.

# 11. Bonnes pratiques et limites

## Bonnes pratiques

- Versionner `.pre-commit-config.yaml`.
- Documenter l'installation des hooks pour chaque développeur.
- Exécuter `pre-commit run --all-files` en CI.
- Utiliser `.gitignore` pour les secrets et fichiers locaux.
- Conserver un fichier `.env.example` sans valeur secrète.
- Examiner les résultats avant de déclarer un faux positif.
- Mettre à jour les hooks de façon contrôlée avec `pre-commit autoupdate`.
- Tester les corrections et conserver une trace des vulnérabilités traitées.
- Prévoir une procédure d'exception limitée, justifiée, approuvée et réexaminée.

## Limites

Les hooks ne remplacent pas :

- La revue de code.
- Les tests fonctionnels et de sécurité.
- Les analyses de dépendances et de conteneurs plus complètes.
- La gestion des secrets en production.
- Les contrôles de CI et les règles de protection des branches.
- L'analyse du risque métier et la validation humaine.

L'objectif n'est pas de multiplier les outils, mais de sélectionner des contrôles adaptés aux risques et de vérifier qu'ils apportent une valeur réelle.

# 12. Indicateurs de suivi

Pour rattacher ce TP au pilotage de la sécurité, proposez des indicateurs simples :

| Indicateur | Ce qu'il mesure |
|---|---|
| Taux de dépôts configurés | Part des dépôts concernés disposant de hooks versionnés. |
| Taux de contrôles CI réussis | Part des exécutions CI terminées sans échec. |
| Nombre de secrets détectés | Nombre de détections à qualifier, sans les assimiler automatiquement à des secrets réels. |
| Nombre de vulnérabilités de dépendances ouvertes | Volume de vulnérabilités restant à traiter après qualification. |
| Délai médian de correction | Temps entre la détection et la vérification de la correction. |

| | |
|---|---|
| Nombre d'exceptions actives | Nombre de dérogations encore autorisées et à réexaminer. |

Pour chaque indicateur, précisez la source, le propriétaire, la fréquence de calcul, le seuil d'alerte éventuel et l'action déclenchée.

Un nombre élevé de détections n'indique pas nécessairement une dégradation : il peut aussi refléter une meilleure couverture des contrôles. Les indicateurs doivent être interprétés avec leur contexte.

# 13. Questions de compréhension

1. Qu'est-ce qu'un hook `pre-commit` ?
2. Quelle différence existe-t-il entre un contrôle local et un contrôle CI ?
3. Pourquoi faut-il versionner `.pre-commit-config.yaml` ?
4. Pourquoi `detect-secrets` peut-il produire des faux positifs ?
5. Pourquoi un baseline ne doit-il pas servir à ignorer des secrets réels ?
6. Quel risque présente l'utilisation de `eval()` sur une entrée non fiable ?
7. Pourquoi un fichier `.env` doit-il être exclu du dépôt ?
8. Pourquoi le fait de déplacer un secret dans une variable d'environnement ne suffit-il pas à sécuriser sa gestion ?
9. Quelle différence existe-t-il entre une analyse de code et un audit des dépendances ?
10. Pourquoi faut-il exécuter les tests dans la CI même si un hook local existe ?
11. Comment empêcher réellement les pushes directs sur `main` ?
12. Quels indicateurs permettraient de suivre l'efficacité du dispositif ?

# 14. Livrables attendus

À la fin du TP, le groupe doit remettre :

- Le dépôt Git avec le fichier `.pre-commit-config.yaml`.
- Le fichier `.gitignore` et le fichier `.env.example`.
- Le code Python corrigé.
- Les tests unitaires.
- Le fichier `requirements.txt` et, si utilisé, `requirements-dev.txt`.
- Le résultat d'une exécution complète des contrôles.
- Le workflow CI. (optionnel)
- Une analyse d'au moins deux problèmes détectés et de leur correction. (optionnel)
- Une courte politique de sécurisation du code.
- Une proposition de trois indicateurs de suivi.